

ABC457 C - Long Sequence 题解

题意概括

给定 N 个序列 $A_1, A_2, \dots, A_N$, 以及对应的重复次数 $C_1, C_2, \dots, C_N$ 。

按照顺序把：

- A_1 追加 C_1 次
- A_2 追加 C_2 次
- ...
- A_N 追加 C_N 次

得到一个很长的序列 B 。要求输出 B_K 。

解题思路

不能真的把整个序列 B 构造出来，因为它的长度可能非常大。

另外，虽然每个序列长度不同，但题目保证所有 L_i 的总和不超过 2×10^5 ，所以原始序列 A_i 本身可以先用普通数组存下来。

我们仍然把所有 A_i 的元素按输入顺序压进一个大数组中：

- $a[p]$ 表示压平后第 p 个位置上的数
- $st[i]$ 表示第 i 个序列在大数组中的起始位置
- $l[i]$ 表示第 i 个序列的长度
- $c[i]$ 表示第 i 个序列要重复多少次

这样，第 i 个序列中的第 j 个数就是：

$$a[st[i] + j - 1]$$

关键观察是：

- 第 i 组在 B 中一共贡献了 $C_i \times L_i$ 个元素

所以我们可以按组判断 K 落在哪一组里。

设当前看到第 i 组：

- 如果 $K > C_i \times L_i$ ，说明第 K 个元素不在这一组里
- 那么就把 K 减去这一整组的长度，继续看下一组

否则，说明答案就在第 i 组中。

接下来还要找出：

- 第 i 组内部的第几个位置是答案

因为第 i 组其实就是把同一个序列 A_i 重复了 C_i 次，所以组内位置是循环的：

- 第 1 个位置对应 $A_{i,1}$
- 第 2 个位置对应 $A_{i,2}$
- ...
- 第 L_i 个位置对应 A_{i,L_i}
- 第 $L_i + 1$ 个位置又回到 $A_{i,1}$

因此，组内位置可以用取模计算：

$$pos = (K - 1) \bmod L_i + 1$$

最后输出 $A_{i,pos}$ 即可。

而在数组里取这个值时，对应的位置就是：

$$a[st[i] + pos - 1]$$

正确性说明

第 i 组由序列 A_i 连续重复 C_i 次组成，因此它在总序列 B 中恰好占据连续的 $C_i \times L_i$ 个位置。

从前往后扫描各组时：

- 如果 K 大于当前整组长度，那么第 K 个元素一定不在这一组中
- 把 K 减去这一组长度后，新的 K 就表示“在剩余后缀中的第几个元素”

重复这个过程后，第一次出现 $K \leq C_i \times L_i$ 的组，一定就是包含答案的那一组。

在这一组内部，由于 A_i 被完整重复，所以组内第 $1, 2, \dots, L_i$ 个位置分别对应 $A_{i,1}, A_{i,2}, \dots, A_{i,L_i}$ ，之后按长度 L_i 循环。

因此，组内第 K 个位置对应的下标就是：

$$(K - 1) \bmod L_i + 1$$

再结合压平数组中的起始位置 $st[i]$ ，输出的就是：

$$a[st[i] + ((K - 1) \bmod L_i + 1) - 1]$$

它恰好对应总序列中的第 K 个元素，所以算法正确。

复杂度

- 时间复杂度: $O(N + \sum L_i)$
- 空间复杂度: $O(\sum L_i)$

这里的 $\sum L_i$ 就是把每个原始序列的长度全部加起来。

参考实现

```
#include<bits/stdc++.h>
using namespace std;

const int MAXN = 200000 + 5;

int a[MAXN];
int st[MAXN];
int l[MAXN];
long long c[MAXN];

int main(){
    // 读取序列个数和目标位置
    int n;
    long long k;
    cin >> n >> k;

    // tot 表示大数组当前已经存了多少个数
    int tot = 0;

    // 读入所有序列
    for(int i=1; i<=n; i++){
        cin >> l[i];

        // 记录第 i 个序列在大数组中的起始位置
        st[i] = tot + 1;

        for(int j=1; j<=l[i]; j++){
            tot++;
            cin >> a[tot];
        }
    }

    // 读入每个序列的重复次数
```

```
for(int i=1; i≤n; i++){
    cin >> c[i];
}

// 逐组判断第 k 个元素落在哪一组中
for(int i=1; i≤n; i++){
    long long block_len = c[i] * l[i];

    if(k > block_len){
        k -= block_len;
    } else {
        // 计算这一组内部对应到 A_i 的哪个位置
        int pos = (int)((k - 1) % l[i] + 1);
        cout << a[st[i] + pos - 1];
        break;
    }
}

return 0;
}
```